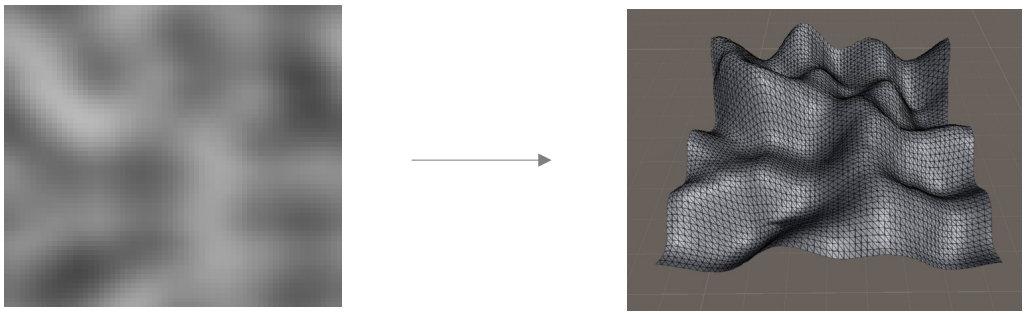
TP – Génération de mesh

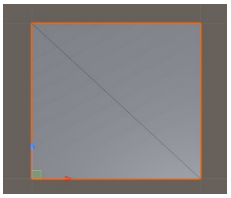
I – Objectif :

L'objectif de ce TP est de créer un mesh de terrain à partir d'une image. Chaque pixel de l'image en niveau de gris correspondra à une position de vertex et à une hauteur.



Dans un nouveau projet Unity3D suivre les étapes suivantes.

II – Création d'un mesh simple de quad :



Créer un script Quad. Ajouter ce script à un GameObject vide.

Ajouter un MeshFilter et un MeshRenderer au GameObject portant le script. Assigner un material standard sur le MeshRenderer.

Au démarrage (Start) du script, déclarer une variable Vector3[] positions = new Vector3[4] contenant les positions des 4 vertices du quad. Renseigner les valeurs suivantes :

- [0] : {0,0,0}
- [1] : {0,0,1}
- [2] : {1,0,0}
- [3] : {1,0,1}

Déclarer une variable de type int[] triangles = new int[3*2] contenant les associations de vertices des deux triangles du quad :

- 1er triangle : {0,1,2}
- 2ème triangle : {2,1,3}

Une fois le tableau entièrement rempli :

- Créer un nouveau Mesh : Mesh mesh = new Mesh(),
- Associer le mesh créé dans la propriété MeshFilter.mesh du MeshFilter,
- Assigner la propriété mesh.indexFormat à UnityEngine.Rendering.IndexFormat.UInt32,
- Assigner la propriété mesh .vertices avec le tableau de vertices,
- Assigner la propriété mesh .triangles avec le tableau d'index de vertices.
- Appeler les méthodes mesh.RecalculateBounds, mesh.RecalculateNormals, et mesh.RecalculateTangents .

III – Mise en place projet :

Télécharger sur internet une image de bruit de Perlin en niveau de gris.

Créer un script TerrainGenerator dans lequel sera développé le projet. Ajouter ce script à un GameObject vide.

Ajouter un MeshFilter et un MeshRenderer au GameObject portant le script d'initialisation de terrain. Assigner un material standard sur le MeshRenderer.

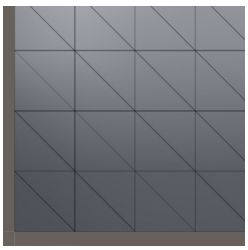
IV – Lecture de l'image :

Au démarrage (Start) du script, récupérer les dimensions width et height de la Texture2D correspondant à l'image de bruit.

Utiliser Texture2D.GetPixels() pour lire les pixels de l'image et stocker les valeurs de couleurs lues dans un tableau de couleurs. Cette fonction renvoie l'ensemble des pixels de la texture dans un tableau aplati à une dimension (pixels de gauche à droite et de bas en haut).

Implémenter une méthode Index2DTo1D permettant de convertir des coordonnées de pixel i et j en un index de ce tableau à une dimension : index = i + j * width

V – Création de mesh :

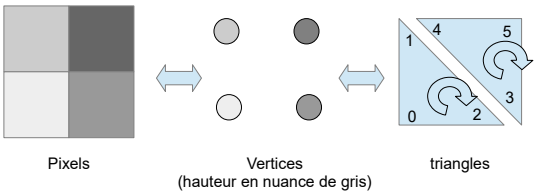


Notre terrain sera constitué de quads.

Chaque pixel de notre texture représentera un vertex du terrain.

- La position du pixel déterminera sa position X et Z
- La couleur du pixel déterminera sa hauteur sur l'axe Y.

Quatre pixels adjacents permettront ainsi de définir un quad, dit « tuile » :

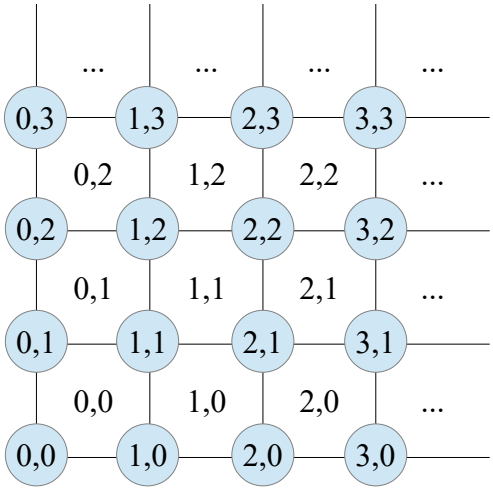


Commençons par générer un mesh sans se soucier de la hauteur des vertices. Dans cette partie, la position d'un vertex sera donc P(x, 0, z).

Pour une texture de dimension (width, height), le terrain sera donc composé de vertexCount = (width * height) vertices formant tileCount = ((width - 1) * (height - 1)) tuiles.

On distinguera par la suite trois systèmes de coordonnées :

- La position 3D d'un vertex dans l'espace : x, y, z
- La coordonnée en index 2D d'un vertex sur le terrain : i, j
- La coordonnée en index 2D d'une tuile sur le terrain : ti, tj



Coordonnées en index de vertices et de tuiles

Définir une variable de taille de tuile.

En utilisant cette variable, implémenter une méthode Index2DToPosition convertissant des coordonnées en index 2D de vertex, (i, j), en position (x, 0, z). La coordonnée 2D du vertex (0, 0) en bas à gauche sera en position (0, 0, 0).

A la suite du code de lecture de texture, déclarer une variable de type Vector3[] contenant toutes les positions des vertices (une par pixel).

Pour chaque vertex d'index (i, j) :

- Calculer la position associée (x, 0, z) en utilisant la fonction Index2DToPosition.
- Calculer l'index 1D du vertex dans le tableau en utilisant Index2DTo1D.
- Renseigner la position du vertex à son index 1D dans le tableau.

Déclarer une variable de type int[] contenant les associations de vertices en triangles composant les tuiles. Chaque tuile est composée de 2 triangle et chaque triangle de trois vertices. La taille du tableau est donc de soit (tileCount * 2 * 3) valeurs.

Soit (ti, tj) l'index 2D d'une tuile, alors les triangles sont composés des vertices avec index 2D suivants :

- 1er triangle : (ti, tj), (ti, tj + 1), (ti + 1, tj)
- 2ème triangle : (ti + 1, tj), (ti, tj + 1), (ti + 1, tj + 1)

Récupérer pour chaque vertex son index associé avec Index2DTo1D et le renseigner dans le tableau d'index de vertices déclaré précédemment pour constituer les triangles.

Une fois le tableau entièrement rempli :

- Créer un nouveau Mesh : Mesh mesh = new Mesh(),
- Associer le mesh créé dans la propriété MeshFilter.mesh du MeshFilter,
- Assigner la propriété mesh.indexFormat à UnityEngine.Rendering.IndexFormat.UInt32,
- Assigner la propriété mesh .vertices avec le tableau de vertices,
- Assigner la propriété mesh .triangles avec le tableau d'index de vertices.
- Appeler les méthodes mesh.RecalculateBounds, mesh.RecalculateNormals, et mesh.RecalculateTangents .

Tester.

VI – Hauteur :

Définir une variable de hauteur maximale d'un vertex.

Ajouter à la composante P.y de la position de chaque vertex la valeur de la couleur (r, g ou b) du pixel multipliée par la variable de hauteur maximale.

Tester.

VII – Texturing :

Implémenter une méthode convertissant des coordonnées des coordonnées de pixel i et j en coordonnées (u,v) linéaires entre 0 et 1. La position de pixel (0, 0) en bas à gauche sera en coordonnées (0, 0).

Dans le script de création de mesh, déclarer une variable de type Vector2[] contenant une coordonnée de texture pour chaque vertex.

Remplir la variable déclarée pour chaque vertex avec les coordonnées calculées par la méthode implémentée ci-dessus.
Assigner les coordonnées de texture dans la propriété mesh.uv du mesh avant les opérations de « Recalculate ».

Assigner une texture principale (albedo) au material du MeshRenderer.

Tester.

VIII – Couleurs :

Ajouter une variable publique de Gradient sur le script.

Dans le script de création de mesh, déclarer une variable de type Color[] contenant une donnée de couleur pour chaque vertex.

En fonction de la couleur du pixel et de la valeur correspondante dans le Gradient, définir la couleur de chaque vertex.

Assigner les données de couleur dans la propriété mesh.colors du mesh avant les opérations de « Recalculate ».

Note : Le material sur le MeshRenderer doit prend en compte la couleur des vertices. Utiliser le shader fourni comme base pour ce material.

Tester.

IX – Fine-tuning :

Ajouter une variable définissant la hauteur du niveau de la mer.

Lors du calcul de position d'un vertex, si la valeur de gris est « sous le niveau de la mer », forcer cette valeur de gris au niveau de la mer.

Télécharger une texture de gradient radial (*radial gradient white to black*) de même taille que la texture de terrain.

Lire les données de texture du gradient et utiliser ces valeurs pour abaisser le relief sur les bords de la carte :

```
noiseValue *= gradientValue ;
```